HARKATI RAYANE 12413993

Rapport TP1 Principe De Programmation

Objectif du TP : Développement d'un service web en utilisant Java.

Le but est de comprendre comment fonctionne un service web, comment il est décrit avec le **WSDL** et comment un client peut communiquer avec ce service.

SOAP : Un service web SOAP permet à des applications de communiquer entre elles via le réseau.
Les données échangées sont sous forme de **XML** et les messages passent par le protocole **HTTP**.

URL : permet d'accéder au service web après son déploiement.

Différence entre SOAP et REST :

SOAP utilise uniquement le format XML et est plus strict.

REST est plus simple et peut utiliser JSON.

SOAP est plus lourd mais plus structuré.

REST est plus léger et plus utilisé aujourd'hui

L'URL suivante : /?wsdl permet d'afficher le fichier WSDL. Ce fichier décrit les méthodes du service web ainsi que les paramètres d'entrée et de sortie.

Il est utilisé pour consommer et invoquer le service web.

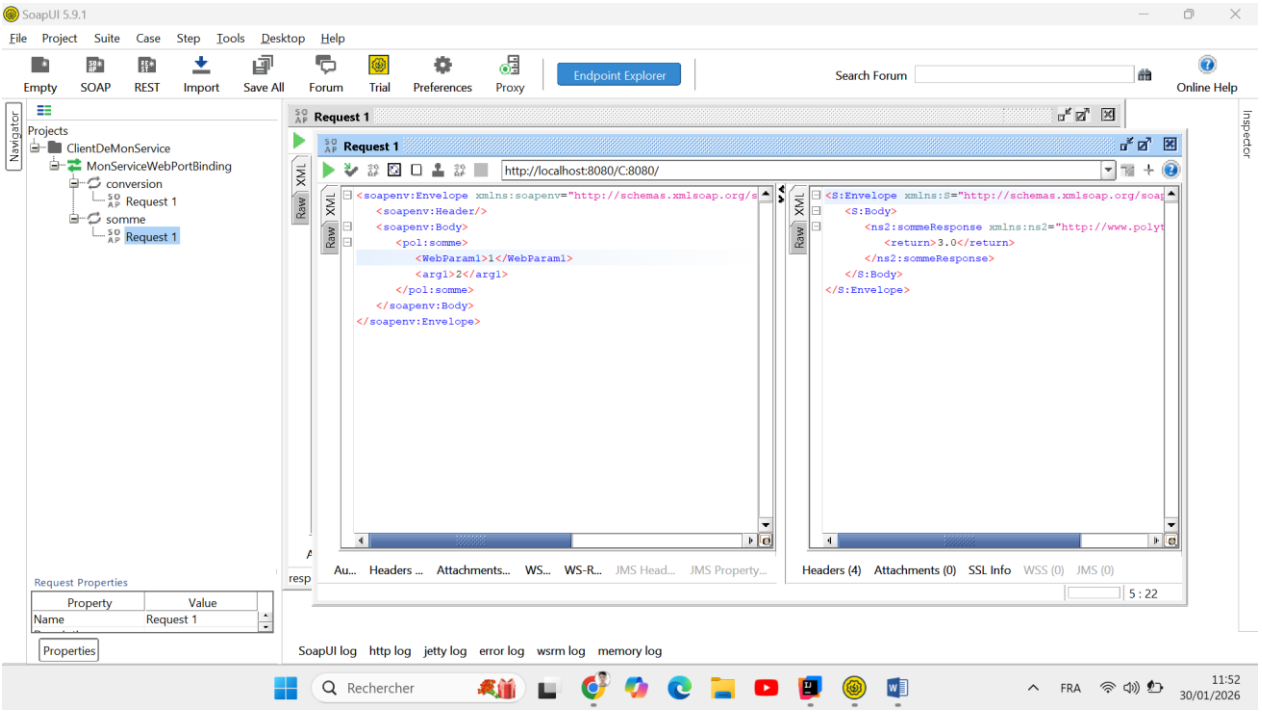
Générer un proxy : À partir du fichier WSDL, on peut générer un **proxy client**.
Ce proxy permet d'appeler les méthodes du service web comme des méthodes Java normales.

Utiliser l'outil SOAPUL : pour créer un client et qui communique avec le serviceWeb et envoie une requête ; Il permet d'envoyer des requêtes SOAP et de voir les réponses reçues du serveur.

Après l'ajout de la fonction `somme`, le service est **redéployé** afin que la nouvelle méthode soit disponible pour les clients.

?/xsd=1 : afficher les différentes opérations offertes par ce services ; permet d'afficher le schéma XSD qui décrit les types de données et les opérations proposées par le service.

HARKATI RAYANE 12413993



Espace de nommage :

Permet d'identifier les éléments XML du service web et d'éviter les conflits de noms.

```
@WebService(targetNamespace = "http://www.polytech.fr")  1 usage
public class MonServiceWeb {
    @WebMethod(operationName = "convertir")  no usages
    public double conversion (double mt) {
        return mt*0.9 ;
    }
    public double somme (@WebParam(name = "WebParam1")double a, double b) {  no usages
        return a+b;
    }
}
```

Partie 2 : Envoyer des objects

Échange des données : Entre le client et le serveur, les données sont échangées sous forme de XML à l'intérieur des messages SOAP

Envoi d'objets Java :Pour envoyer des objets :

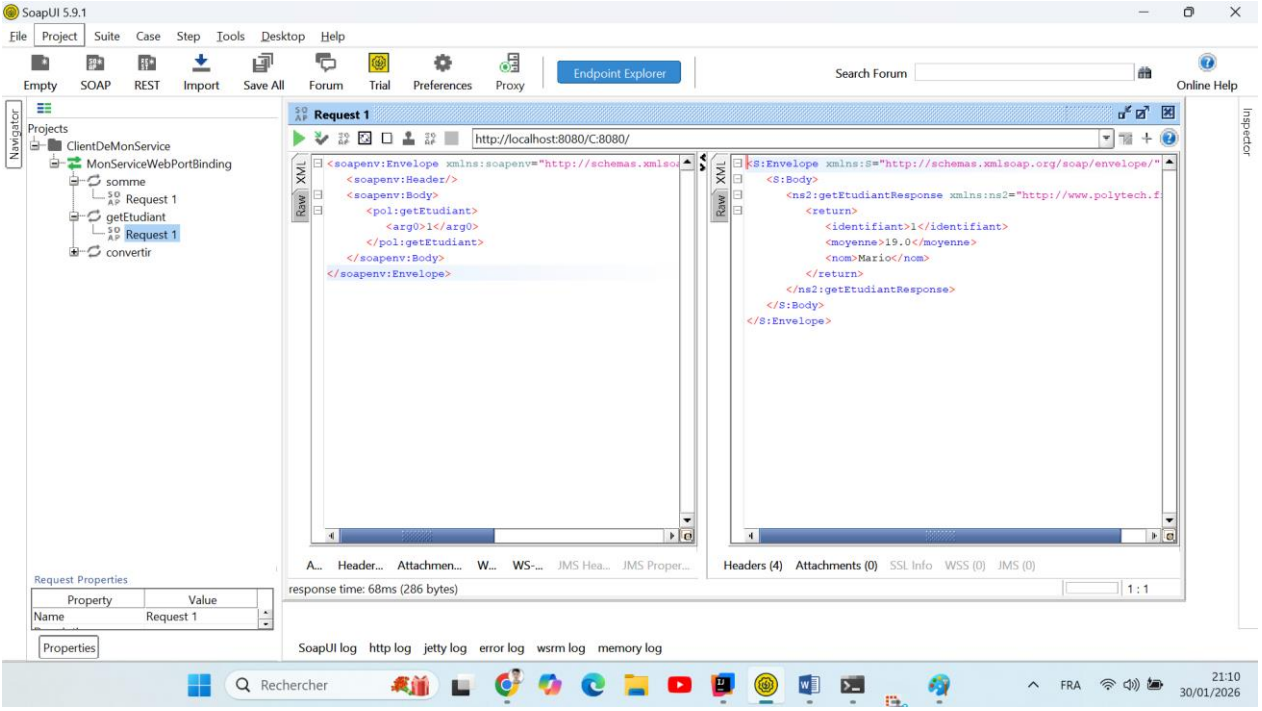
on utilise un **Bean** Java : c'est **une classe Java simple** qui sert à **stocker des données** et à les échanger entre différentes parties d'une application (client / serveur, service web, etc.).,

la classe doit implémenter Serializable,

la classe doit être annotée avec @XmlRootElement.

HARKATI RAYANE 12413993

Sérialisation : permet de transformer un objet Java en XML pour l'envoyer dans un message SOAP, puis de le reconstruire côté serveur.

Conclusion

Ce TP m'a permis de comprendre le fonctionnement des services web SOAP, l'utilisation du WSDL, la génération de proxy et l'échange d'objets entre un client et un serveur.